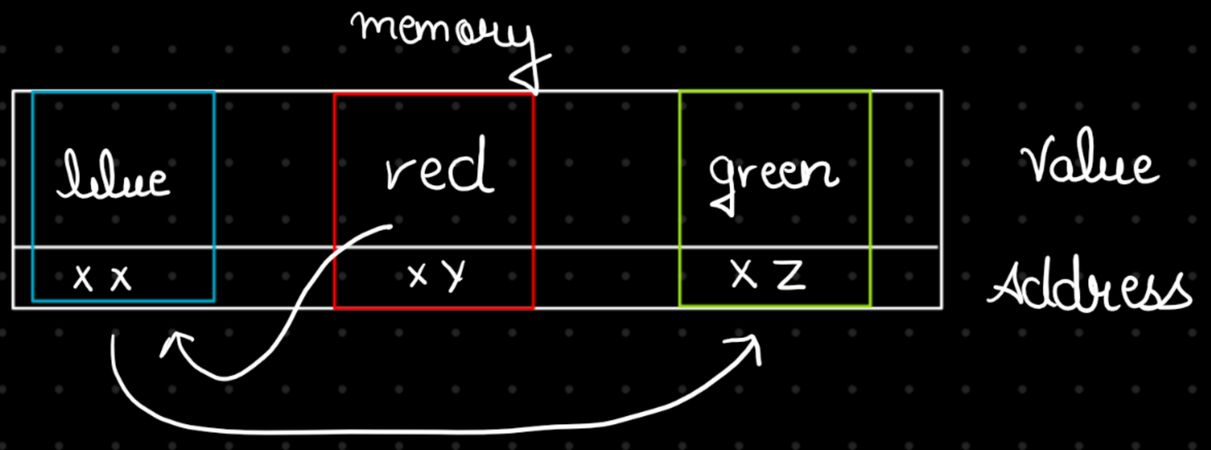
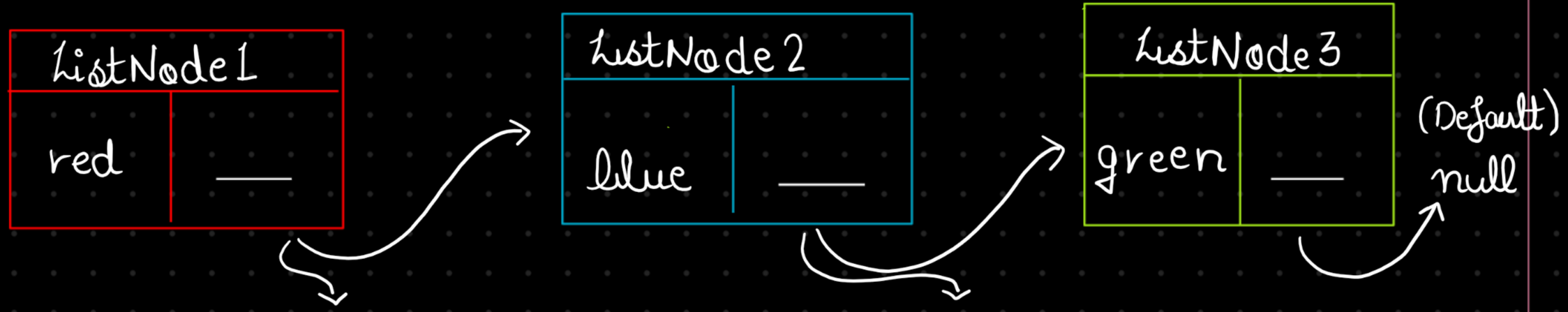
Linked list

↳ each listNode/Node contain atleast two things

→ Pointer to next Node





listNode 1 . next = ListNode 2

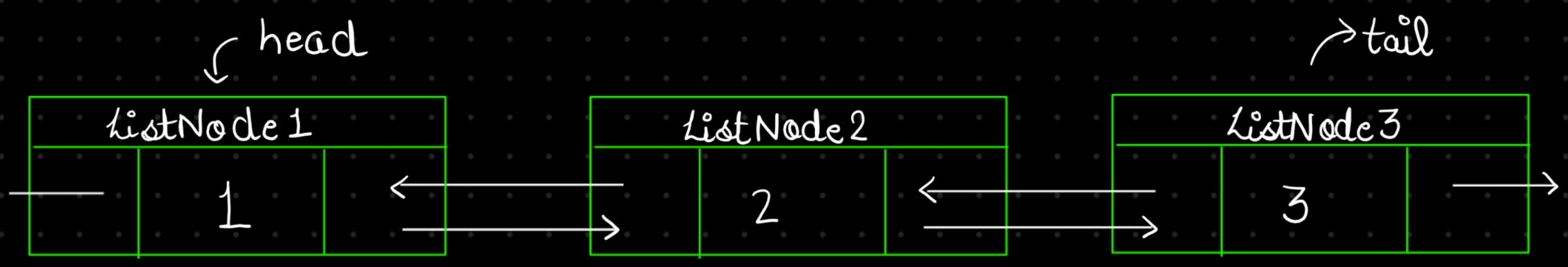
listNode 2 . next = ListNode 3

Tips →

1) Alway have variable pointing to the head and tail of a linked list for easy insertion.

Doubly linked list

↳ each list contains three things.



Implementation

1) Singly linked list

```
# include <iostream>
using std :: cout
using std :: endl;
class listNode {
public :
    int val_;
    listNode * = nullptr;
    listNode ( int val){
        val_ = val ;
    }
};
class linkedlist {
public:
    listNode * head;
    listNode * tail;
    linkedList ( ){
        // Init the list with a 'dummy' node which makes
        // removing a node from the beggining of list easier.
        head = new listNode(-1 );
```

```
        tail = head;
    }

    void insertEnd(int val){
        tail->next = new ListNode(val);
        tail = tail->next;
    }

    void remove(int index){
        int i = 0;
        ListNode * curr = head;
        while( i < index && curr ){
            i++;
            curr = curr->next;
        }
        // Remove the node ahead of curr
        if( curr && curr->next){
            if( curr->next == tail ){
                tail = curr;
            }
            curr->next = curr->next->next;
        }
    }
```

```
void print(){
    ListNode * curr = head->next; // skiping dummy
    while(curr){
        cout << curr->val_ << " ->";
        curr = curr -> next;
    }
    cout << endl;
    }
};
```

Doubly linked list

```
#include <iostream>
using std:cout;
using std: endl;

class LinkedList {
public:
    int val_;
```

```
class linkedlist{
public:
    ListNode * next;
    listNode * tail;
    linkedlist( ) {
        head = new listNode(-1);
        tail =  new ListNode(-1);
    }
void insertFront( int val ) {
    ListNode * newNode = new listNode(val);
    newNode -> prev = head;
    newNode -> next = head -> next;
    head -> next -> prev = newNode;
    head -> next = newNode;
}
void insertEnd( int val) {
    ListNode * newNode = new
```